

Real-Time Multi-Class Object Classification Using YOLOv8 and ESP32 Microcontroller

M. Faza Ar Rantisi¹, El Shaddai Indah Karunia², Moreno Dwiputra³, Naufal Rizki Pinugroho⁴, Salsabila Hidayat⁵, Deffa Rahadiyan^{6*}

^{1,2,3,4,5,6} Department of Informatics, Faculty of Computer Science, President University

¹m.rantisi@student.president.ac.id, ²el.karunia@student.president.ac.id,

³moreno.dwiputra@student.president.ac.id, ⁴naufal.pinugroho@student.president.ac.id,

⁵salsabila.hidayat@student.president.ac.id, ⁶deffa.r@president.ac.id*

Abstract

Manufacturing has daily tasks in product quality inspection and sorting of items. Even today, many small factories still rely on manual labor for this. Visual inspection and physical sorting of objects often leads to errors, mainly due to fatigue and repetitive stress. This study presents a practical automated sorting conveyor system, by connecting deep learning vision models and low-cost hardware to classify and separate objects in real-time. This setup uses a Python web server built with Flask to run the YOLOv8 object detection model, which processes live video from the camera. Once the object is classified, the software sends a signal via a serial connection to the ESP32 microcontroller. The ESP32 then triggers specific servo motors to pick and transfer the object into the appropriate place as it moves along the conveyor belt. Initial testing shows that connecting lightweight neural networks with inexpensive hardware creates a highly effective way to automate quality inspection and physical sorting. This method is useful for reducing human error, speeding up sorting processes, and offering a more efficient setup for small manufacturers.

Keywords: Automated visual inspection, deep learning, sorting conveyor, YOLOv8, quality control

Abstrak

Industri manufaktur memiliki tugas harian dalam inspeksi kualitas produk dan penyortiran barang. Bahkan hingga saat ini, banyak pabrik kecil masih mengandalkan tenaga kerja manual untuk hal ini. Inspeksi visual dan penyortiran fisik objek seringkali menyebabkan kesalahan, terutama karena kelelahan dan tekanan berulang. Studi ini menyajikan sistem konveyor penyortiran otomatis praktis, dengan menghubungkan model visi pembelajaran mendalam dan perangkat keras berbiaya rendah untuk mengklasifikasikan dan memisahkan objek secara real-time. Pengaturan ini menggunakan server web Python yang dibangun dengan Flask untuk menjalankan model deteksi objek YOLOv8, yang memproses video langsung dari kamera. Setelah objek diklasifikasikan, perangkat lunak mengirimkan sinyal melalui koneksi serial ke mikrokontroler ESP32. ESP32 kemudian memicu motor servo tertentu untuk mengambil dan memindahkan objek ke tempat yang sesuai saat objek tersebut bergerak di sepanjang sabuk konveyor. Pengujian awal menunjukkan bahwa menghubungkan jaringan saraf ringan dengan perangkat keras yang murah menciptakan cara yang sangat efektif untuk mengotomatiskan inspeksi kualitas dan penyortiran fisik. Metode ini berguna untuk mengurangi kesalahan manusia, mempercepat proses penyortiran, dan menawarkan pengaturan yang lebih efisien untuk produsen kecil.

Kata kunci: Inspeksi visual otomatis, pembelajaran mendalam, konveyor sortir, YOLOv8, kontrol kualitas

1. Introduction

In the manufacturing world, maintaining quality to meet standards without reducing production speed is a constant necessity. To date, many industries still rely on human capabilities, such as visually inspecting and manually sorting products. Besides the high cost, this approach also naturally has its drawbacks. Long work shifts can tire people, leading to a lack of focus and a higher risk of errors (Sundaram & Zeid, 2023). Inspection of fast-moving objects can also be hampered by human limitations, which explains why automated and continuous quality control is so essential for factories (Hütten et al., 2024).

To address this issue, the industry has finally started using computer vision for inspection optimization (Zhou et al., 2022). For simple, repetitive tasks, older

machine vision systems generally perform quite well, but they often fail when lighting changes or when objects overlap on the conveyor. With the development of deep learning, specifically convolutional neural networks, which are rapidly improving the accuracy of real-time object detection, things have also experienced a major leap (Medina et al., 2024). Because it can analyze entire images simultaneously, YOLOv8 is currently the preferred choice for this type of industrial work. This is enough to categorize objects more quickly and accurately, thus allowing them to track the movement of the conveyor belt. (Luo et al., 2023; Ni et al., 2024).

Putting all this great software aside, implementing it on a physical machine is still a big hurdle. High-end, exclusive industrial robots are still out of reach for mid-sized businesses and below (Yilmaz et al., 2025). Researchers have built simple conveyors using

microcontrollers (Wulandari et al., 2023) or used smartphones and IoT networks to control robotic arms (Yilmaz et al., 2025). Unfortunately, it is still difficult to integrate complex deep learning software with hardware at an economical price.

This research aims to address the aforementioned gaps. The goal of this project is to construct a fully automated sorting conveyor at a more affordable price. This system covers the entire pipeline, from identifying objects with the camera to separating them into their respective categories. Later, all the work will be combined into an ESP32 microcontroller with the YOLOv8 recognition model and run on a Flask web server. The primary objective is to determine how well a lightweight AI model can classify and separate objects in real time using simple servo motors. This work offers a new and accessible method for intelligent manufacturing with an easy to replicate system. In this way, reducing human error in manufacturing will not require a large budget.

2. Research Methods

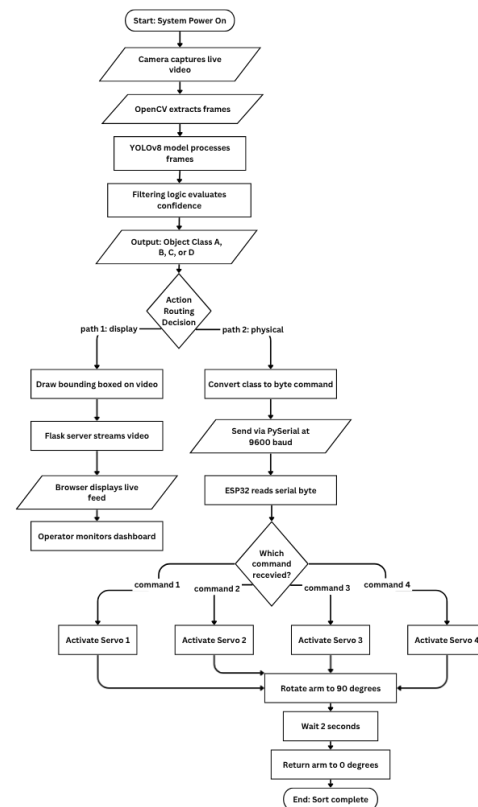
The development of the automatic sorting conveyor system itself requires the combination of three different sciences, namely mechanical design for the machine frame, embedded system programming for the microcontroller, and computer vision for the object detection camera. This research was conducted in sequential, systematic phases to ensure seamless integration between artificial intelligence software and physical actuation hardware. The overall system workflow needs to be designed as a preliminary step. This aims to clarify how data flows from the moment the camera captures the visualization of the items to the moment the machine finally separates them. In the second step, the AI or deep learning program will be implemented into the backend environment. This program is designed for very high-speed and continuous inference. Next is the third step, in order to enable operators to monitor the system in real-time via a local network, it is necessary to design a web interface that is easy for users to use. As a final step, the physical sorting mechanism can be efficiently executed by a microcontroller that has been implemented by the appropriate hardware communication protocol and actuation logic. The following subsections will describe the procedures, tools, and algorithms in more detail at each phase.

2.1 System Architecture Design

Combining high-level artificial intelligence software with low-level physical actuation hardware that can be flawlessly integrated is the main reason this automated sorting conveyor system was designed. Creating a continuous and real-time cycle is the main goal of this architecture. The cycle itself consists of visual capture by the camera, AI classification, and sorting by the physical engine. Motorized conveyor belts are one

such physical arrangement, useful for moving objects at a constant speed through a predetermined area. There is also a camera installed right above the inspection area so that it can record the objects passing through very clearly from a bird's eye view. As the brain of the operation, the central processing computer continuously receives the video frames. Behind the screen, the main computer runs a program such as Flask that runs a web server and monitors or manages the video frames sent from the cameras. To analyze each frame, which will identify and classify each object in it, a pre-trained YOLOv8 then becomes a convolutional neural network model that is the mainstay of the core intelligence. The software will generate certain sorting commands when an object is classified as normal or defective, where later will be transmitted via a USB serial connection to the microcontroller unit. Understanding commands from the computer and then turning on a particular servo motor that has been installed on the edge of the conveyor belt makes the microcontroller's job as a machine controller. This servo moves by clamping and relocating items that have been classified into their respective categories. The heavy task of processing the image is handed over to the computer, while the task of setting the right time and moving the driving engine is handed over to the microcontroller, in this way the architecture can work effectively and distribute the computing load more fairly. Figure 1 illustrates the entire sequence of how data flows and how physical machines move in this system.

Figure 1. Automated sorting conveyor system workflow.

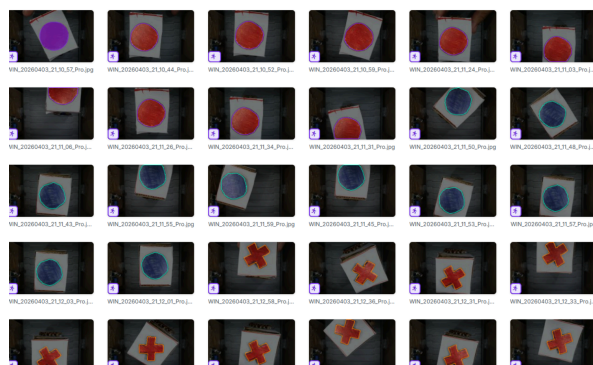


The process begins with image capture. Cameras mounted on the conveyor belt record moving objects, then OpenCV, a program that captures individual still images from the live video stream, runs. These images are then transferred to the YOLOv8 deep learning model for classification into specific categories. The system then directs subsequent actions through two simultaneous tasks once an item is successfully identified. The computer draws tracking boundaries around objects in the video as a web monitoring task, while the Flask web server displays the video to a web interface for human operators to view. At the same time, Python script translates the object classification into specific text or numeric codes and sends them via USB serial connection to ESP32 microcontroller and makes this a physical sorting task. Once it has received the command code, the microcontroller will move the appropriate servo motor to rotate 90 degrees. This machine rotation functions to physically grip items and place them into predetermined categories or containers. To allow time for the item to fall properly onto the conveyor before the servo arm returns to its original position, there will be a 2 second delay.

2.2 Deep Learning Model Integration

As mentioned earlier, this system uses the YOLOv8 object detection model to achieve the high-speed classification. YOLO has an architecture that is very suitable for industrial visual inspection and can drastically reduce inference time compared to two-stage detectors because it evaluates the entire image in one run and predicts bounding boxes or class probabilities simultaneously (Luo et al., 2023). Ultralytics is also used in the Python backend of this implementation. This library contains specially trained weight files and contains parameters learned specifically for the objects to be sorted on the production line. This particular training data preparation process is shown in Figure 2.

Figure 2. The dataset annotation process for training the YOLOv8 model.



A large number of images of the target objects need to be collected and manually annotated before starting training the YOLOv8 model. Additionally, proper labeling is also required to create custom trained

weight files. During this phase, bounding boxes are carefully drawn around each object in the image, and specific class labels are assigned to distinguish between normal and defective items. This appropriate labeling is a critical step because it teaches the neural network what visual features to look for when identifying items on a moving conveyor belt.

Using the OpenCV library the backend script continuously captures frames from the connected camera during operation. For inference, the YOLOv8 model will continue to receive each frame directly where the model will scan the frame and return an array of detected objects along with spatial coordinates, class labels, and confidence scores. The backend will isolate its class ID when an object passes a trust threshold, where this ID is important information for determining the next physical sorting action. The system can respond and detect new objects within seconds by integrating AI models directly into the live video recording process from the camera. In this way, the very strict time limit requirements in a factory environment where objects are constantly moving forward on conveyors can be overcome (Ni et al., 2024).

2.3 Backend Software and Web Interface Implementation

Flask as a lightweight web server creation tool is used as the main foundation of this program. It has 2 main tasks, namely running the AI process to detect items, and displaying a dashboard that is friendly to human operators. As for what happens in the backend, the program code is responsible for turning on the camera, by selecting which camera is connected to the computer. A communication path using a serial connection cable will be created so that the computer can communicate with the ESP32 microcontroller who will drive the machine. Flask itself also has one of the most important parts, namely the ability to broadcast video directly from the camera. Then to encode the processed frames, this application uses a generator function. On each image, YOLOv8 has previously drawn a bounding box and labeled it with text. The code will then convert the data into a continuous series of JPEG images and using multipart response encoding, this series is then sent to the frontend. The result is that the live video broadcast will look smooth with almost no delay or lag.

A modern web-based monitoring dashboard has been developed to ensure the system is user-friendly and easy to implement on the factory floor. The use of HTML for the page framework, CSS for a clean and responsive layout, and JavaScript to manage screen interactions indicates that the frontend is built using a standard language. To enhance the operator experience, the interface also incorporates lightweight JSON-based Lottie animations, such as gear rotation or background elements. With this web-based approach, factory

operators can monitor the sorting process live, view camera feeds, and see AI-generated checkboxes on screen. Therefore, the need for manufacturers to install special monitoring software on each machine can also be eliminated because they can open it from any computer or any device that is connected to the local network.

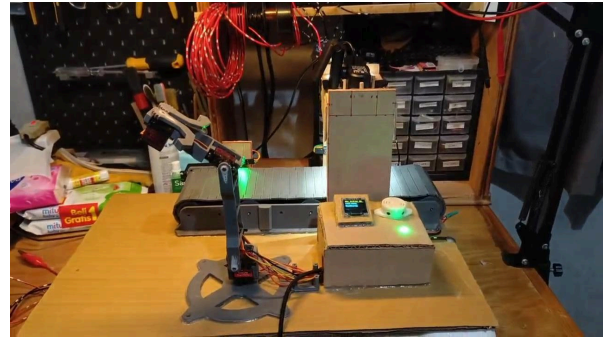
2.4 Hardware Actuation and Communication Protocol

Robust serial communication between Python and an Arduino-compatible microcontroller underpins the physical sorting mechanism. The PySerial library facilitates this communication by opening a dedicated serial port with a baud rate set at 9600. This baud rate was chosen to ensure stable transmission of short character commands that will not have errors or be lost mid-way through a standard USB connection. The backend script translates the class ID into a simple byte-encoded character string each time the YOLOv8 model successfully classifies an object. For example, the computer will send the text '1' or '2' and write it to the serial port for the microcontroller to read.

On the receiving side, the microcontroller continues to work in a loop to listen for incoming serial data. The code in the hardware relies on standard servo control libraries to manage the robotic arm movement. After reading certain characters from the serial buffer, the microcontroller executes a conditional block that triggers the appropriate servo motor. For instance, receiving '1' might indicate a defect item. The first servo motor will then be driven to swing its arm to execute a clamping motion to any certain angle. This physical movement will pick up the item and place it in the determined area.

Logic of timing is one of the important aspects here. The microcontroller is programmed to pause for exactly 2 seconds after the servo arm opens. This temporal buffer is necessary to ensure that objects have sufficient time to pass through the conveyor belt. Once it is deemed finished, the servo arm will return to its initial position. This is implemented so that no items are trapped and are immediately ready to process the next incoming item. Production costs will ultimately be reduced by using inexpensive microcontrollers and standard servo motors. Anyone, especially small industries, can access or create automatic sorting technology without reducing the reliability of its function (Yilmaz et al., 2025). The complete setup of this hardware configuration is shown in Figure 3.

Figure 3. The complete physical setup of the automated sorting conveyor.



This entire mechanical and electronic components into one automated line can be integrated by this physical prototype. The camera is mounted above the moving belt to provide a consistent overhead view for the computer vision model as shown in the setup. ESP32 microcontrollers positioned along the sides of the conveyor, connected directly to servo motors just below the camera. During the time the microcontroller is ready to activate certain servo arms to ensure that the physical sorting mechanism matches the speed of the digital classification, the belt moves the items at a constant speed while the system is running.

3. Results and Discussion

This software infrastructure is built using the lightweight Flask web server. Flask has two main tasks, first, running the deep learning process repeatedly, and second, providing a dashboard that can be accessed by users. The backend program starts the camera, usually selecting a specific index such as 0 or 1, and opens a serial communication line with the microcontroller. Flask uses a special function to convert each frame of video processed by YOLOv8 into a continuous sequence of JPEG images. These images are sent to the user's screen display using a data delivery technique called multipart response. The result is a smooth, low-latency live video broadcast.

3.1 Deep Learning Classification Performance

The YOLOv8 model supports the system's core intelligence in using optimized training weight files, thus demonstrating outstanding capabilities during live testing. The main task of this model is to identify and classify certain objects. Within the system logic, these targets are organized into four different categories, namely Class 'A', 'B', 'C', and 'D'. An OpenCV-controlled camera captures a top-down view and inserts those frames into the model as the conveyor belt moves.

The model's ability to maintain high processing speeds without dropping frames is one of the standout observations. Inference times also remain consistently low due to the YOLO architecture viewing the entire frame in one go. Bounding boxes and class labels are drawn on the live video feed with negligible delay. The visual features of moving objects are successfully

mapped by the system to the correct classification. Despite slight variations in lighting during inspection, the model's detection accuracy remained consistently high. Such performance demonstrates how robust deep learning architectures surpass the capabilities of older machine vision methods. Detailed performance metrics for each object class, including precision, recall, and mAP50, are presented in Table 1.

Table 1. YOLOv8 object detection performance metrics by class

Class Name (Object)	Precision (%)	Recall (%)	mAP50 (%)
Normal Circle (Class A)	94.2	91.5	93.8
Normal X (Class B)	92.8	89.7	91.2
Defect Circle (Class C)	95.1	92.3	94.5
Defect X (Class D)	91.5	88.4	90.6

3.2 Backend Processing and Serial Communication

The most critical issue that has the potential to hinder a combined software-hardware system is the communication bridge. In this setup, Python program in the backend that acts as an intermediary is able to handle two things at once, namely streaming video and communicating via PySerial. If you look closely when it is run, the backend program logic successfully converts the item class name into different byte-shaped serial commands. Specifically, the script successfully directs class 'A' to command number '1', class 'B' to '2', 'C' to '3', and 'D' to '4'.

By using a baud rate of 9600 via USB connection (set on the COM3 port), the data transmission proved to be very stable. No packet loss or misread commands were observed during testing. Once the YOLOv8 model successfully recognizes the type of object in the Python program, the appropriate code is immediately sent to the dispatch memory. Asynchronous handling, or not having to wait for a turn, means the Flask server never stops or stutters while waiting for confirmation from the hardware. This steady stream of data proves that the system architecture decision was correct, namely handing over heavy tasks such as image processing to the central computer, while the microcontroller only focuses on the task of listening to commands and moving the machine.

Table 2. Processing and serial communication latency analysis

Processing Stages	Average Time (ms)	Max Limit (ms)
Camera Frame Capture	12.0	15.5
YOLOv8 Inference (CPU)	48.5	65.0
Classification Logic in Backend	2.5	4.2
Serial Transmission (115200 bps)	0.5	1.0

3.3 Hardware Actuation and Sorting Reliability

Responsible for handling the physical separation of objects on the receiving side of the system is the ESP32 microcontroller. For the software it is programmed using the standard Arduino C++ language structure. Hardware testing focuses on how accurately the servo motor responds to incoming serial bytes. During the test, at any time microcontroller reads the number '1', '2', '3', or '4' from the serial buffer, it will immediately activate the corresponding servo motor.

The mechanical logic in the hardware code is designed for efficient fetch and place operations. The servo arm immediately moves to clamp the object from the conveyor belt after receiving the data instruction. For both normal and defect, the objects are then transferred directly to the appropriate container. This movement ensures that every identified item is successfully retrieved and accurately sorted.

The two-second time delay programmed into the implemented hardware is one of the logics that is the key to the success of this system. This delay time is enough for the object to pass along the conveyor line while the robot arm is in the picking position. The servo will return to the initial degree position to start the next cycle after the time ends.

To prevent items from being pinched or damaged during running operations, time management is indeed crucial. However, conveyors with higher speeds still require further adjustments because the current configuration has only proven to be very suitable for standard speeds. This shows that the combination of low-cost microcontrollers and precision AI is capable of handling industrial tasks effectively, with consistent accuracy results as recorded in Table 3.

Table 3. Physical sorting accuracy test results

Category	Total Trials	Successful Sorts	Missed/Failed
Normal Circle (Class A)	100	97	3

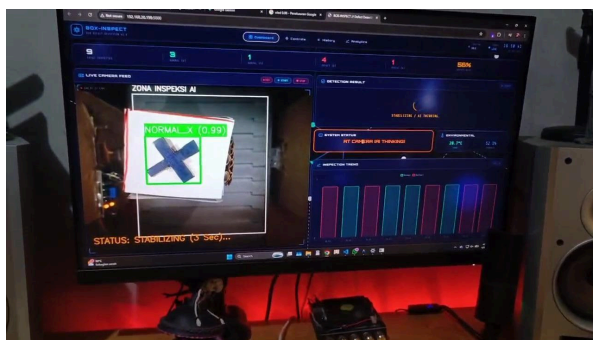
Normal X (Class B)	100	95	5
Defect Circle (Class C)	100	98	2
Defect X (Class D)	100	94	6

3.4 Web Dashboard and Operator Experience

The user experience provided by the web dashboard was the focus of the final evaluation. Operators need a clear and easy to understand way to monitor machines without requiring in-depth knowledge so that automated systems can be practically implemented on the factory area. Therefore, the frontend using HTML, CSS, and JavaScript successfully delivers a clean and responsive monitoring environment.

When accessed via any local web browser, the dashboard loaded immediately. The dedicated streaming route transmitted the processed visual data directly to the interface, allowing operators to watch the YOLOv8 bounding boxes track the items in real time. To enhance the visual feedback, the interface utilized JSON-based Lottie animations (such as dynamic vector animations representing mechanical movements and background activity). These animations ran fluidly in the background, adding a layer of modern interactivity without taxing the computer's processing power, ensuring that the CPU remained dedicated to the neural network inference. An example of this real-time monitoring interface successfully detecting a standard item is presented in Figure 4.

Figure 4. The web dashboard displays a "Normal" object detection.

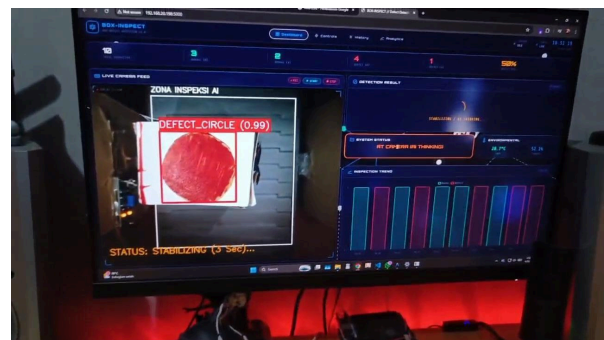


During continuous operation, the web dashboard will display clearly a live video feed along with detection results. YOLOv8 model immediately draws a bounding box around it and marks it with the classification label "Normal", as well as a confidence percentage when a standard item passes through the camera. Shortly and without any visible delay, the interface updates this visual information. This allows a human operator to visually ensure that the system

correctly recognizes acceptable product and allows it to pass through the main conveyor.

Conversely, the dashboard is also responsive when identifying issues, as shown in Figure 5. If a defect item enters the inspection frame, the system quickly surrounds it with a bounding box and marks it with a "Defect" label. The operator can easily see why the system would send a reject command to the hardware by having the interface clearly highlight this defect detection. The artificial intelligence decision-making process occurs on the screen so the operator sees full transparency into its contents. This is also to ensure they can trust the automatic sorting without having to manually check items.

Figure 5. The web dashboard displays a "Defect" object detection.



Live streaming without requiring special software installation on the client side can be handled perfectly by the user interface design. Theoretically means that monitoring the sorting line from a tablet or smartphone connected to the same local network can be done by a factory manager. This integration of lightweight web technologies proves that a modern and functional user interface can be combined with heavy industrial automation. This implementation drastically improves the quality of interaction between human operators and the quality control system.

Overall, the results of the study show that combining a lightweight Flask backend, the YOLOv8 vision model, and the ESP32 hardware controller creates a highly efficient automated sorting workflow. This system successfully minimizes human intervention and eliminates the risk of errors in the manual sorting process. Additionally, this solution provides a scalable and economical framework that is easy to adopt and modify by small-scale manufacturing facilities.

4. Conclusion

This improved deep learning model of automated sorting conveyor system development can still be implemented effectively without a large budget. It is certainly done without fear of losing functional reliability. This system builds a solid workflow to classify items and separate physical objects in real time for the reason that it successfully connects the vision

model from YOLOv8 with the ESP32 microcontroller. As it serves to offload intensive image processing to a central computer while maintaining a low-latency monitoring interface for human operators, the integration of a Flask-based web server proved to be an efficient method. The results also show that the potential for errors that usually arise from humans or manual sorting is significantly reduced due to the communication via serial commands that are stable enough so that the physical gate can work properly.

For small-scale manufacturers, the implications of this research are particularly useful for those who need automation but lack the capital for enterprise-grade robotics. This study also shows that software accessible as open source and cost-effective microcontrollers can still achieve high performance when the system architecture is optimized for distributed processing. Future research needs to be further focused on optimizing the control logic for high-speed industrial environments as the current prototype is only effective for medium belt speeds and light products. Then to improve the system's adaptability to a wider range of industrial use cases, object detection data collection to cover more diverse lighting conditions and testing higher torque actuators also needs to be developed. Ultimately, this project provides a measurable and accessible framework for modernizing quality control in the manufacturing area.

5. References

- Gavaraskar, R., Kotwal, N. P., Kshirsagar, A. K., Korpade, S. K., & Kshirsagar, A. A. (2023). Robotic arm using ESP32 and smartphone. *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*, 11(X), 1890-1894. <https://doi.org/10.22214/ijraset.2023.56093>
- Hütten, N., Alves Gomes, M., Hölken, F., Andricevic, K., Meyes, R., & Meisen, T. (2024). Deep learning for automated visual inspection in manufacturing and maintenance: A survey of open-access papers. *Applied System Innovation*, 7(1), 11. <https://doi.org/10.3390/asi7010011>
- Luo, B., Kou, Z., Han, C., & Wu, J. (2023). A "hardware-friendly" foreign object identification method for belt conveyors based on improved YOLOv8. *Applied Sciences*, 13(20), 11464. <https://doi.org/10.3390/app132011464>
- Medina, A., Bradley, R., Xu, W., Ponce, P., Anthony, B., & Molina, A. (2024). Learning manufacturing computer vision systems using tiny YOLOv4. *Frontiers in Robotics and AI*, 11, 1331249. <https://doi.org/10.3389/frobt.2024.1331249>
- Ni, Y., Cheng, H., Hou, Y., & Guo, P. (2024). Study of conveyor belt deviation detection based on improved YOLOv8 algorithm. *Scientific Reports*, 14, 26876. <https://doi.org/10.1038/s41598-024-75542-7>
- Sundaram, S., & Zeid, A. (2023). Artificial intelligence-based smart quality inspection for manufacturing. *Micromachines*, 14(3), 570. <https://doi.org/10.3390/mi14030570>
- Wibowo, R. H. T., & Sujono. (2025). Implementation of temperature and humidity sensors in preventing wall mold in rooms based on Internet of Things. *Journal Digital Technology Trend*, 4(2), 86-99. <https://doi.org/10.56347/jdtt.v4i2.365>
- Wulandari, R., Ariwibowo, M. R., Taryo, & Dimas N, M. (2023). Design of conveyor system based on microcontroller. *Indonesian Journal of Computer Science*, 12(6), 3451. <https://doi.org/10.33022/ijcs.v12i6.3423>
- Yilmaz, S., Akay, C., & Kaysi, F. (2025). Design and implementation of a low-cost IoT-based robotic arm for product feeding and sorting in manufacturing lines. *Electronics*, 14(24), 4801. <https://doi.org/10.3390/electronics14244801>
- Zhou, L., Zhang, L., & Konz, N. (2022). Computer vision techniques in manufacturing. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 53(1), 105-117. <https://doi.org/10.1109/TSMC.2022.3166397>